

# Riassunto della tesi

Sviluppo di un sistema RAG aziendale e valutazione di LLM locali

Francesco Romeo

Anno Accademico 2025/2026

## Contesto e obiettivo del lavoro

La tesi affronta la progettazione, l'implementazione e la valutazione di un sistema di *Retrieval-Augmented Generation* (RAG) integrato in i3Guild, la piattaforma usata da Intré S.r.l. per gestire le Gilde aziendali. Il punto di partenza è un problema concreto: nel corso degli anni i3Guild ha raccolto proposte, descrizioni, obiettivi, risultati, partecipanti e metadati relativi alle attività di apprendimento interno dell'azienda. Questo patrimonio informativo rappresenta una memoria organizzativa utile, ma non è sempre facile da consultare con strumenti tradizionali, perché le informazioni sono distribuite tra database relazionali, campi testuali, metadati e contenuti collegati.

L'obiettivo del lavoro non è addestrare un nuovo modello linguistico. La tesi studia invece come integrare componenti esistenti in un prototipo enterprise on-premise, capace di rispondere a domande in linguaggio naturale sul corpus delle Gilde. La soluzione deve rispettare tre vincoli principali. Il primo è la riservatezza: i dati aziendali devono rimanere entro un perimetro controllato. Il secondo è la sostenibilità infrastrutturale: il sistema deve poter funzionare su risorse hardware realistiche, senza assumere la disponibilità di server GPU dedicati. Il terzo è la manutenibilità: l'integrazione deve inserirsi nell'applicazione esistente senza confondere sorgente dati, indicizzazione, retrieval e generazione.

Il paradigma RAG è adatto a questo scenario perché separa la conoscenza usata dal sistema dai parametri del modello generativo. Un Large Language Model pre-addestrato può produrre testo coerente, ma non conosce i dati privati e aggiornati dell'organizzazione. In un sistema RAG, invece, la domanda dell'utente viene prima usata per recuperare documenti pertinenti da una base di conoscenza esterna; solo dopo questi documenti vengono inseriti nel prompt del modello, che genera una risposta fondata sul contesto recuperato. Questa impostazione riduce il rischio di risposte scollegate dal corpus e permette di aggiornare la conoscenza intervenendo sull'indice, non sui pesi del modello.

## Fondamenti tecnici e requisiti

La prima parte della tesi introduce i fondamenti necessari per comprendere il prototipo. I Transformer costituiscono la base degli attuali modelli linguistici e dei modelli di embedding.

Il meccanismo di self-attention consente di costruire rappresentazioni contestuali del testo, superando diversi limiti delle architetture ricorrenti. Tuttavia, gli LLM rimangono sistemi a conoscenza chiusa quando sono usati senza strumenti esterni: non hanno accesso nativo ai dati interni, possono generare informazioni plausibili ma errate e non offrono da soli un meccanismo di verifica sulle fonti aziendali.

Da questi limiti deriva la scelta del RAG. La tesi distingue due pipeline: una pipeline di ingestione, eseguita offline, e una pipeline di inferenza, attivata in tempo reale dalle richieste utente. La prima normalizza i contenuti, costruisce documenti adatti al retrieval, genera embedding densi e sparsi e li salva in un database vettoriale. La seconda riceve la query, produce le rappresentazioni della domanda, recupera i documenti più pertinenti e costruisce il prompt aumentato per il modello generativo. In questo modo il sistema può combinare similarità semantica, segnali lessicali e filtri basati sui metadati.

Il caso d'uso scelto riguarda il supporto alla proposta di nuove Gilde. Prima di proporre un nuovo tema, una persona può avere bisogno di sapere se argomenti simili siano già stati affrontati, quali risultati siano stati prodotti, chi abbia partecipato e in quale epoca siano nate iniziative affini. L'analisi del corpus mostra che la Gilda è l'unità informativa più adatta all'indicizzazione: contiene campi descrittivi, obiettivi, rischi, risultati attesi o raggiunti e collegamenti con partecipanti ed epoch. I metadati non vengono trattati come semplice testo, ma restano disponibili per filtrare e contestualizzare le risposte.

I requisiti del sistema riflettono questa struttura. Il prototipo deve recuperare informazioni dal corpus i3Guild, generare risposte in linguaggio naturale, indicare il contesto usato, supportare filtri strutturati e integrarsi con l'interfaccia Next.js esistente. Deve anche evitare dipendenze non necessarie da servizi esterni, perché il caso di studio richiede controllo sui dati e sull'infrastruttura. La scelta tecnologica cade quindi su PostgreSQL come sorgente autoritativa, Qdrant come database vettoriale, Haystack come framework di orchestrazione, FastAPI come microservizio RAG, Ollama per l'inferenza locale e Next.js per l'integrazione applicativa.

## Progettazione e implementazione del sistema

L'architettura proposta separa in modo netto i componenti. PostgreSQL conserva i dati applicativi di i3Guild; una pipeline di embedding legge le informazioni rilevanti, normalizza i campi testuali e costruisce documenti Haystack; Qdrant memorizza embedding densi, rappresentazioni sparse e payload strutturati; il microservizio FastAPI espone retrieval, chat sincrona e chat in streaming; l'applicazione Next.js consuma il servizio tramite client dedicati. Questa suddivisione riduce l'accoppiamento: l'applicazione rimane la sorgente autoritativa, l'indice può essere ricostruito o aggiornato, e il modello generativo può essere sostituito senza riscrivere l'intero sistema.

La pipeline di ingestione è progettata come job batch. Prima controlla la configurazione, la connessione a PostgreSQL, la raggiungibilità di Qdrant e la coerenza dei parametri di embedding. Poi estrae le Gilde con le relative epoch e i partecipanti, pulisce i campi HTML con una normalizzazione testuale e compone un documento strutturato in sezioni riconoscibili. In condizioni ordinarie una Gilda viene indicizzata come documento consolidato, perché la

dimensione media dei campi resta compatibile con la finestra del modello di embedding. Per gli outlier è prevista una soglia di sicurezza: se il testo supera il limite, viene suddiviso in chunk con sovrapposizione controllata. Gli identificativi dei documenti sono deterministici, così una reindicizzazione non duplica i punti in Qdrant ma sovrascrive quelli esistenti.

Il retrieval non si limita a una ricerca vettoriale. La query dell'utente viene normalizzata e, quando possibile, riscritta per isolare il tema effettivo della domanda. Nel dominio i3Guild parole come "gilda", "fondare" o "consigli" possono essere poco informative: nella frase "vorrei fondare una gilda sulla musica", il topic utile per la ricerca è "musica". In parallelo, il sistema prova a riconoscere vincoli espressi in linguaggio naturale, come una specifica epoca, la modalità di partecipazione o la natura individuale o di gruppo della Gilda. Dopo il recupero, i risultati vengono deduplicati e selezionati con soglie dinamiche, evitando di passare al modello documenti troppo simili o poco pertinenti.

La generazione usa Ollama come server locale. Il microservizio costruisce un prompt aumentato con istruzioni, domanda utente e contesto recuperato, poi invoca il modello e restituisce la risposta. Lo streaming NDJSON permette di separare eventi di retrieval, token generati, warning ed errori, migliorando la percezione di reattività nell'interfaccia. Il servizio include anche controlli operativi, come stime di memoria disponibile, gestione dei timeout, rate limiting e distinzione tra errori applicativi e dipendenze non raggiungibili.

Durante lo sviluppo è stato implementato anche un workflow agentic per guidare la proposta di nuove Gilde. La tesi lo documenta come sperimentazione, non come nucleo della soluzione finale. La ragione è pratica: a parità di hardware locale, l'approccio agentic introduce più chiamate al modello, maggiore latenza e maggiore complessità di controllo. Nel perimetro del prototipo, un RAG più classico e deterministico risulta più stabile, più semplice da mantenere e più coerente con i vincoli disponibili. La componente agentic rimane quindi una possibile evoluzione, da valutare solo se produce un beneficio misurabile.

## Valutazione dell'inferenza locale

La seconda parte della tesi affronta la sostenibilità dell'inferenza locale. Un sistema RAG aziendale può essere architetturealmente corretto, ma la sua utilità dipende anche dal modello generativo usato nell'ultimo stadio della pipeline. Il modello deve rientrare nella memoria disponibile, caricarsi in tempi accettabili, iniziare a generare senza ritardi e mantenere una qualità sufficiente per il caso d'uso. Per questo viene definito un protocollo sperimentale dedicato ai modelli open-weight eseguiti localmente su Apple Silicon.

L'esperimento usa un MacBook Pro con chip M1 Pro e 16 GB di memoria unificata. Il runtime scelto è MLX, perché è progettato per Apple Silicon e consente di convertire, quantizzare ed eseguire modelli linguistici in modo coerente con il backend Metal. La matrice include modelli tra 3B e 14B parametri, divisi in classi small, medium e large. Per i modelli piccoli vengono confrontate FP16, Q8, Q6 e Q4; per quelli medi e grandi l'attenzione si concentra soprattutto sulle varianti quantizzate, perché la quantizzazione diventa una condizione di fattibilità entro il vincolo dei 16 GB.

Le metriche misurate sono dimensione su disco, memoria di picco, tempo di caricamento, time-to-first-token, throughput di decodifica e throughput end-to-end. A queste si aggiungono controlli automatici di qualità su subset ridotti MMLU e GSM8K. La scelta di subset piccoli non mira a costruire una classifica generale dei modelli; serve a individuare degradazioni macroscopiche, problemi di formato e incompatibilità con il parser. I fallimenti non vengono rimossi dall'analisi, perché in un sistema reale anche il formato dell'output è parte del comportamento osservabile.

I risultati mostrano che, sui modelli piccoli, Q4 è il compromesso più interessante. Riduce la dimensione degli artefatti di circa il 72%, riduce il picco di memoria di circa 67–70% e aumenta il throughput di decodifica di circa 2.6–2.8 volte rispetto a FP16. Nei controlli automatici ridotti non emerge un degrado macroscopico. Q8 e Q6 sono varianti più conservative, ma nel setup misurato non offrono un vantaggio qualitativo sufficiente a compensare il maggiore uso di memoria.

Per i modelli medi e grandi la lettura cambia. Qwen3 8B e Mistral 7B diventano utilizzabili in Q4 con throughput intorno a 32–35 token al secondo e picchi di memoria inferiori a 5 GB. Gemma 3 12B e Qwen3 14B sono eseguibili in Q4, ma con throughput intorno a 18 token al secondo e time-to-first-token superiore al secondo. Queste configurazioni dimostrano la fattibilità tecnica, ma non sono automaticamente preferibili in un'applicazione interattiva. Il margine residuo di memoria deve infatti essere condiviso con Qdrant, FastAPI, Next.js, sistema operativo e altri processi.

## Risultati, limiti e sviluppi futuri

La tesi conferma che un sistema RAG locale per un contesto aziendale è realizzabile, ma richiede controllo simultaneo di più livelli. Non basta scegliere un modello linguistico: occorre curare il corpus, progettare l'indice, definire la logica di retrieval, controllare prompt e formato della risposta, verificare la sostenibilità dell'inferenza e integrare il tutto con l'applicazione esistente. Il contributo principale del lavoro è quindi un prototipo integrato e valutato in condizioni realistiche, più che una singola innovazione algoritmica.

Dal punto di vista architetturale, il prototipo mostra che i3Guild può essere trasformato in una knowledge base interrogabile mantenendo PostgreSQL come sorgente autoritativa e separando l'indice vettoriale dal database applicativo. Dal punto di vista del retrieval, l'uso congiunto di embedding densi, segnali sparsi, filtri strutturati, deduplicazione e selezione dinamica consente di costruire un contesto più aderente al dominio rispetto a una ricerca puramente semantica. Dal punto di vista sperimentale, la quantizzazione MLX rende praticabili diversi modelli open-weight su hardware consumer Apple, con un profilo particolarmente favorevole per i modelli piccoli in Q4.

Restano alcuni limiti. Il primo riguarda la valutazione applicativa del RAG: il prototipo è stato implementato, ma manca una ground truth estesa costruita con stakeholder aziendali per misurare groundedness, completezza, utilità e allucinazioni sul corpus reale. Il secondo riguarda i dati: le Gilde sono eterogenee per granularità, stile e completezza dei campi. Il terzo riguarda l'esperimento MLX, eseguito su una sola macchina e con subset di qualità ridotti; i risultati non possono essere generalizzati direttamente ad altri hardware o runtime.

Gli sviluppi futuri seguono da questi limiti. La priorità è costruire un dataset di domande realistiche sul dominio i3Guild, con risposte attese e documenti di riferimento validati da persone interne all'azienda. Su questa base si potranno confrontare versioni diverse della pipeline e misurare il contributo effettivo di query rewriting, filtri, soglie e deduplicazione. Un secondo sviluppo riguarda l'indicizzazione incrementale, che potrebbe sostituire la reindicizzazione completa quando il corpus crescerà. Un terzo riguarda la qualità del contesto recuperato, migliorabile con glossari aziendali, sinonimi controllati, tag più espliciti e, in prospettiva, reranker.

La componente agentica potrà essere ripresa solo dopo il consolidamento del RAG di base. Un agente potrebbe aiutare nella proposta di nuove Gilde, nella raccolta di requisiti o nel collegamento con iniziative passate, ma la sua adozione dovrebbe dipendere da criteri misurabili: riduzione del lavoro per l'utente, qualità superiore delle bozze, controllo dei costi computazionali e robustezza del formato prodotto. Nel perimetro attuale, la direzione più utile non è aumentare la complessità dell'orchestrazione, ma rafforzare il legame tra corpus, retrieval, valutazione e vincoli reali dell'infrastruttura locale.